

CS 423

Operating System Design: Swapping/Intro to Concurrency

Sep 29

Jiyu Hu

Slide ack: Prof. Shivaram Venkataraman (Wisconsin)

ANNOUNCEMENTS

MP2 released

Next lecture: MP2 walkthrough by TAs

AGENDA / LEARNING OUTCOMES

Finish the discussion on swapping

A pop-quiz

Start on concurrency (second piece):

- What is the motivation for concurrent execution

- What are some challenges?

RECAP

Swap Space

Designated & reserved fraction of persistent storage

- Sizing is configurable; usually multiple of the physical memory

Persistent storage is accessed in units of “blocks”

- Typically, block size is equal to or a multiple of page size.
- “Paging out”: write a page out to swap space
- “Paging in”: read a page in from swap space*

OS tracks free space in the swap space

- Simple block addressing: linearly from 0 to n

*not necessarily

Virtual Address Space Mechanisms

Basic mechanism: use present bit to say whether page is in physical memory

If not present, PTE typically points to location on swap device

Unless its code/static data (backed by the binary)

On access, HW raises an exception called page fault

OS handles the page fault

Bring in the required page from swap space (or the binary)

Modifies the PTEs to reflect these changes

Swap/page daemon creates free space based on two watermarks (low and hi)

If victim-page has dirty bit set, it needs to be written out swap space

Else it can be reclaimed immediately

Interaction With OS Scheduler

During TLB miss handling, process is in _____ state

(Hard) page faults are expensive: OS must read from disk

During page fault handling, process is in _____ state

Once page fault handling kicks off the read I/O, process moves to _____ state

The process moves to _____ state once the read I/O completes and sets the PTE's present bit.

END RECAP

SWAP SPACE LOCATION

The fact that swap space is persistent is somewhat *incidental*

It happens to be the next tier in the memory hierarchy!

Persistence allows features, such as laptop hibernation

Smartphones typically use zRAM instead

Local: SSD or hard drive or USB device

Remote: storage on a different system

RDMA-based swapping

SWAPPING POLICIES

SWAPPING Policies

Goal: Minimize number of page faults

- Page faults require millisecs to handle (reading from swap)
- Implication: Lots of time for OS to make good decisions

OS has two decisions

- **Page selection**
When should a page (or pages) on disk be **brought into** memory?
- **Page replacement**
Which resident page (or pages) in memory should be **kicked out**?

Page Selection

Demand paging: Load page only when page fault occurs

- Intuition: Wait until the last minute; when page is absolutely needed
- When a process starts up: No pages are loaded in memory
- Problem: Pay cost of page fault for every newly accessed page

Pre-paging (anticipatory, prefetching): Load page beforehand

- OS predicts future access and prefetches pages into memory
- Works well for some access patterns (e.g., sequential)
- Two costs of bad prefetching?

Some combination of the two, together with hints

Hints: User-supplied hints about page references

Hints with `madvise()`

`int madvise(void *addr, size_t length, int advice)` - allows user to give hints to OS

`MADV_RANDOM` // kernel may disable prefetching

`MADV_SEQUENTIAL` // kernel may aggressively prefetch...

 // ...and kick out after access

`MADV_WILLNEED`

`MADV_DONTNEED`

and more...

Generally, hard to tune and extract better performance than OS default

Page Replacement

One potential dimension for selecting victim pages kick out:

- Write out victim page to disk if modified (dirty bit set)
- If victim page is not modified (clean), just discard
- So when it makes sense, **pick clean pages as victims**

Now the policies

OPTIMAL: Replace the page to be accessed furthest in the future

- Pro: Guaranteed to minimize #page faults
- Con: Requires knowing the future! Impractical, but good for comparisons

Page Replacement

FIFO: Replace page that has been in memory the longest

- Intuition: Referenced long time ago, get rid of it
- Advantage: Fair as each page gets equal residency; Easy to implement
- Disadvantage: Some pages may remain popular

LRU: Least-recently-used

- Intuition: Recent past predicts the near future
- Advantages: With locality, LRU approximates OPTIMAL
- Disadvantages:
 - Harder to implement, must track which pages have been accessed

Page Replacement - chat for 2 mins

Page reference string: ABCABDADBCB

Metric:
Miss count

Three pages
of physical
memory

	OPT	FIFO	LRU									
ABC	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
A	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
B	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
D	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
A	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
D	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
B	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
C	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			
B	<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>				<table><tr><td></td><td></td><td></td></tr></table>			

Page Replacement

Page reference string: ABCABDADBCB

Metric:
Miss count

Three pages
of physical
memory

	OPT	FIFO	LRU									
ABC	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C
A	B	C										
A	B	C										
A	B	C										
A	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C
A	B	C										
A	B	C										
A	B	C										
B	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C	<table><tr><td>A</td><td>B</td><td>C</td></tr></table>	A	B	C
A	B	C										
A	B	C										
A	B	C										
D	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D	<table><tr><td>D</td><td>B</td><td>C</td></tr></table>	D	B	C	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D
A	B	D										
D	B	C										
A	B	D										
A	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D	<table><tr><td>D</td><td>A</td><td>C</td></tr></table>	D	A	C	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D
A	B	D										
D	A	C										
A	B	D										
D	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D	<table><tr><td>D</td><td>A</td><td>C</td></tr></table>	D	A	C	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D
A	B	D										
D	A	C										
A	B	D										
B	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D	<table><tr><td>D</td><td>A</td><td>B</td></tr></table>	D	A	B	<table><tr><td>A</td><td>B</td><td>D</td></tr></table>	A	B	D
A	B	D										
D	A	B										
A	B	D										
C	<table><tr><td>C</td><td>B</td><td>D</td></tr></table>	C	B	D	<table><tr><td>C</td><td>A</td><td>B</td></tr></table>	C	A	B	<table><tr><td>C</td><td>B</td><td>D</td></tr></table>	C	B	D
C	B	D										
C	A	B										
C	B	D										
B	<table><tr><td>C</td><td>B</td><td>D</td></tr></table>	C	B	D	<table><tr><td>C</td><td>A</td><td>B</td></tr></table>	C	A	B	<table><tr><td>C</td><td>B</td><td>D</td></tr></table>	C	B	D
C	B	D										
C	A	B										
C	B	D										

LRU vs. OPT

QUESTION: Think of a workload & config where LRU will be way worse than OPT. Random replacement might do better than LRU!

Page Replacement Comparison

What happens to performance with increase in physical memory size?

LRU, OPT:

- Guaranteed to have fewer (or same number of) page faults
- Stack property:
 - cache of size $N+1$ includes contents of cache of size N

FIFO:

- Usually have fewer page faults
- Belady's anomaly: May actually have **more** page faults!

FIFO PERFORMANCE MAY DECREASE!

Consider access stream: ABCDABEABCDE

Physical memory size: 3 pages vs. 4 pages

How many misses with FIFO?

Page Replacement

Page reference string: ABCDABEABCDE

	3 pages			4 pages			
ABC							
D							
A							
B							
E							
A							
B							
C							
D							

Page Replacement

Page reference string: ABCDABEABCDE

	3 pages			4 pages			
ABC	A	B	C	A	B	C	
D	B	C	D	A	B	C	D
A	C	D	A	A	B	C	D
B	D	A	B	A	B	C	D
E	A	B	E	B	C	D	E
A	A	B	E	C	D	E	A
B	A	B	E	D	E	A	B
C	B	E	C	E	A	B	C
D	E	C	D	A	B	C	D
E	E	C	D	B	C	E	E

FIFO PERFORMANCE MAY DECREASE!

Consider access stream: ABCDABEABCDE

Physical memory size: 3 pages vs. 4 pages

How many misses with FIFO?

9 misses with 3 pages and 10 misses with 4 pages

3 Pages

A, B, C, D, A, B, E miss

A hit, B hit, C, D miss

E hit

4 pages

A, B, C, D miss

A hit, B hit, E miss –

A, B, C, D, E miss

IMPLEMENTING LRU

Software Perfect LRU

- OS maintains ordering of physical pages by reference time
- When page is referenced: modify the ordering
- When need victim: Pick page with oldest time
- Trade-off: Slow on memory reference, fast on replacement

Hardware Perfect LRU

- Associate timestamp with each page
- When page is referenced: Store system clock with page
- When need victim: Scan to find oldest timestamp
- Trade-off: Fast on memory reference, slow on replacement (especially as size of memory grows)

In practice, do not implement Perfect LRU

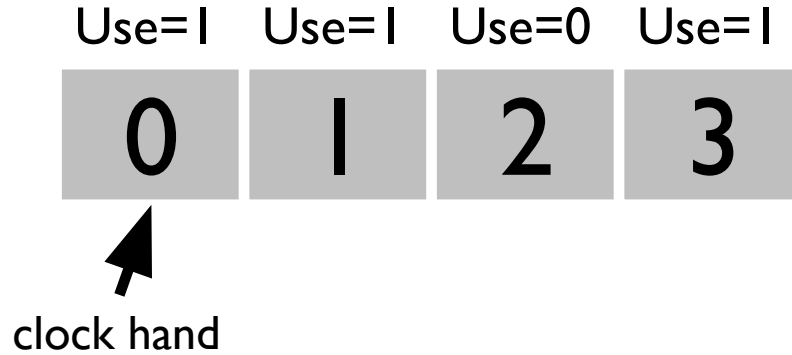
- LRU is an approximation anyway, so approximate some more :-)
- Goal: Find an old page, but not necessarily the very oldest

CLOCK ALGORITHM: APPROX LRU

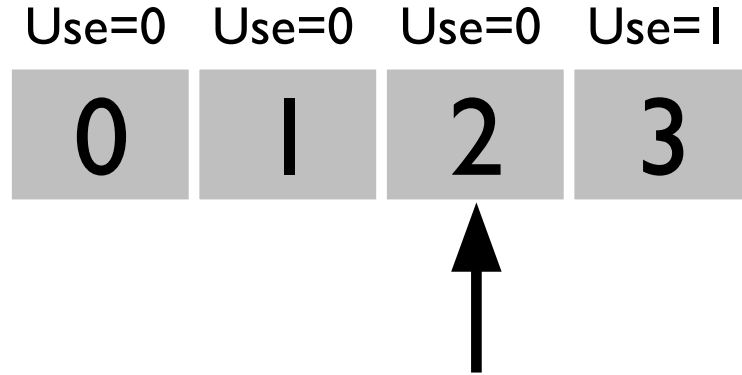
- PTE comprises a use/reference bit
- Is set whenever a page is referenced
- Page replacement: Look for page with use bit cleared (has not been referenced for awhile)
- Implementation:
 - Keep pointer to last examined page frame
 - Traverse pages in circular fashion
 - Clear use bits as search
 - Stop when find page with already cleared use bit, replace this page

CLOCK: LOOK FOR A PAGE

Physical Mem:



Evict
contents of
frame 2, load
in new page



CLOCK EXTENSIONS

Replace multiple pages at once

- Intuition: Efficient to write pages to swap space in bulk
- Find multiple victims each time and track free list

Use dirty bit to bias against replacing dirty pages

- Intuition: More expensive to replace dirty pages
Dirty pages must be written to disk, clean pages do not
- Replace pages that have use bit and dirty bit cleared

Add software counter (“unused”)

Intuition: Can tell how often pages are accessed

Increment software counter if use bit is 0

Replace when change exceeds some specified limit

LINUX 2Q

Linux has a unified page cache (will cover later in course)

Maintains active list vs inactive list

On first reference, put page in inactive list

- If accessed again, promoted to active list

- Pick victim pages from inactive list in FIFO order to swap out

In background: move pages from active to inactive list until active list is $\leq$ 2/3rd of physical memory size

- Protects against larger-than-cache-cyclic-access pattern (will cover later)

LINUX CODE

`include/linux/swap.h`

Data structure used to track swap area

`struct swap_info_struct`

Operates in units of 256 pages “clusters”

`struct swap_cluster_info`

Parallelism: each CPU is given a cluster

OUT OF MEMORY

What happens when total memory allocated to all processes > RAM + swap space?

1. OS constantly pages in/out from swap space—aka *thrashing*

ProcessA kicks out p2 to page in p1, ProcessB kicks out p1 to page in p2, and so on...

System noticeably slows down

2. OOM Killer (kernel) gets invoked

Kill processes that hog memory; typically younger processes

3. User processes: malloc() returns null

SUMMARY: VIRTUAL MEMORY

Abstraction: Virtual address space with code, heap, stack

Address translation

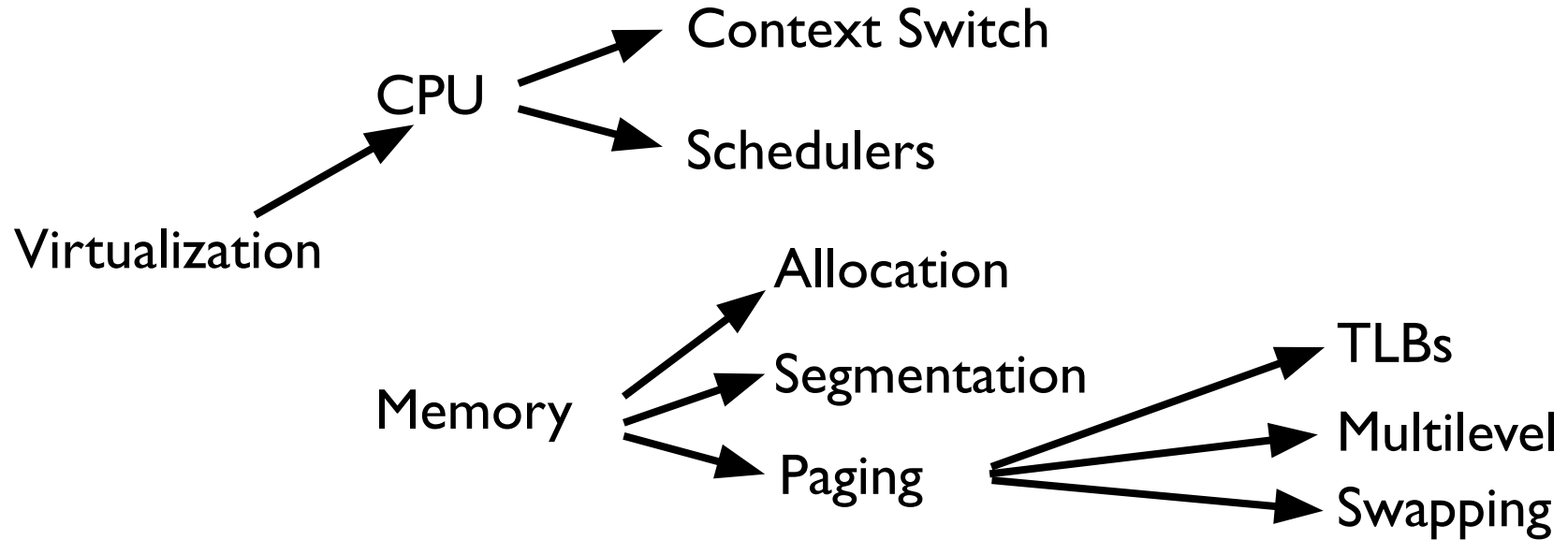
- Contiguous memory: base, bounds, segmentation
- Using fixed sizes pages with page tables

Challenges with paging

- Extra memory references: avoid with TLB
- Page table size: avoid with multi-level paging, inverted page tables etc.

Larger address spaces: Swapping mechanisms, policies (LRU, Clock)

REVIEW: EASY PIECE 1



Next: Concurrency

Concurrency:

What is the motivation for concurrent execution

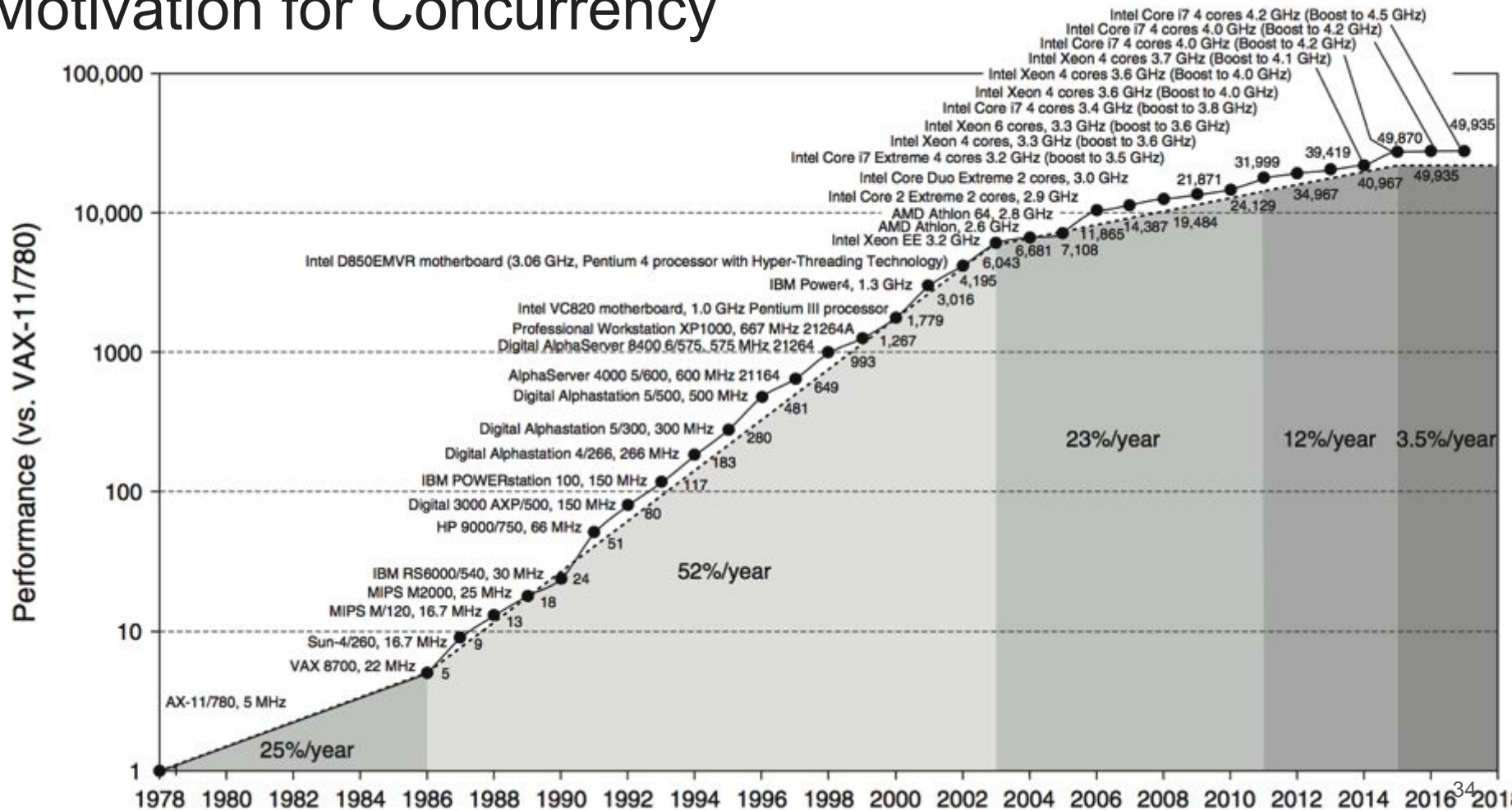
Challenges?

Threading & Locks

For folks that have taken 341 (or its equivalent)

Some of this will be obvious; but we'll dig deeper

Motivation for Concurrency



MOTIVATION

CPU Trend: Speed has stabilized; increasing (maybe differentiated) #cores

Goal: Application utilize more cores

Option 1: Build app with many **processes**

- Example: Browser (process per tab)

Pros:

- Don't need new abstractions
- Good for security

Cons:

- High communication overhead
- Expensive context switching

Option 2: Build app with many **threads**

- Example: Word processors, video games

Pros:

- Shared address space
- Cheaper: fast context switch

Cons:

- Data corruption

Many modern apps are hybrid: use both options

THREADING

**Threads (of same process)
share address space**

**Each thread has its “own”
stack**

“own” -> but not really
protected from other threads

Question: Walk through eg. of a
bug where a thread corrupts
another thread's stack.

THREADING

New abstraction: thread

Threads are like “lightweight processes”, except:

Multiple threads (of same process) share the address space

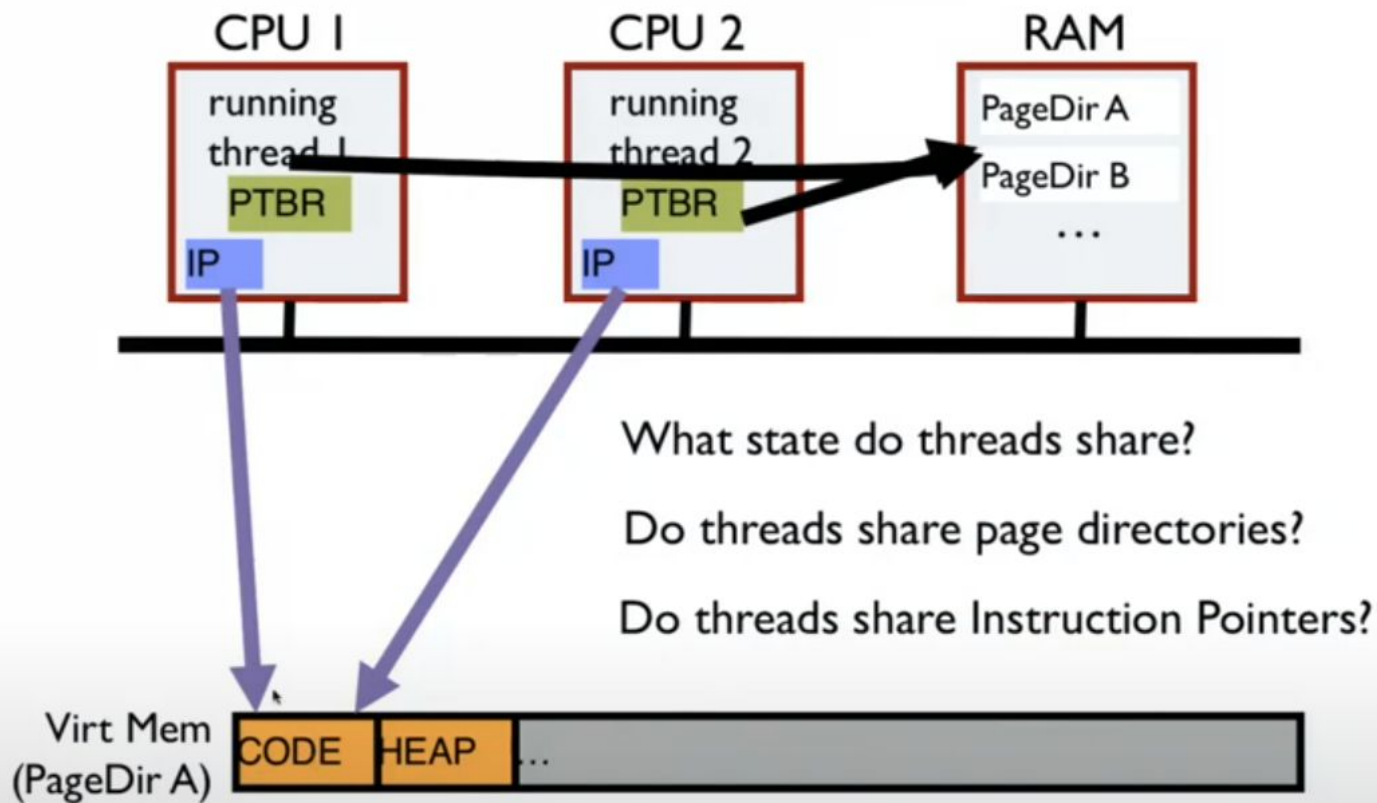
Each thread has its “own” stack

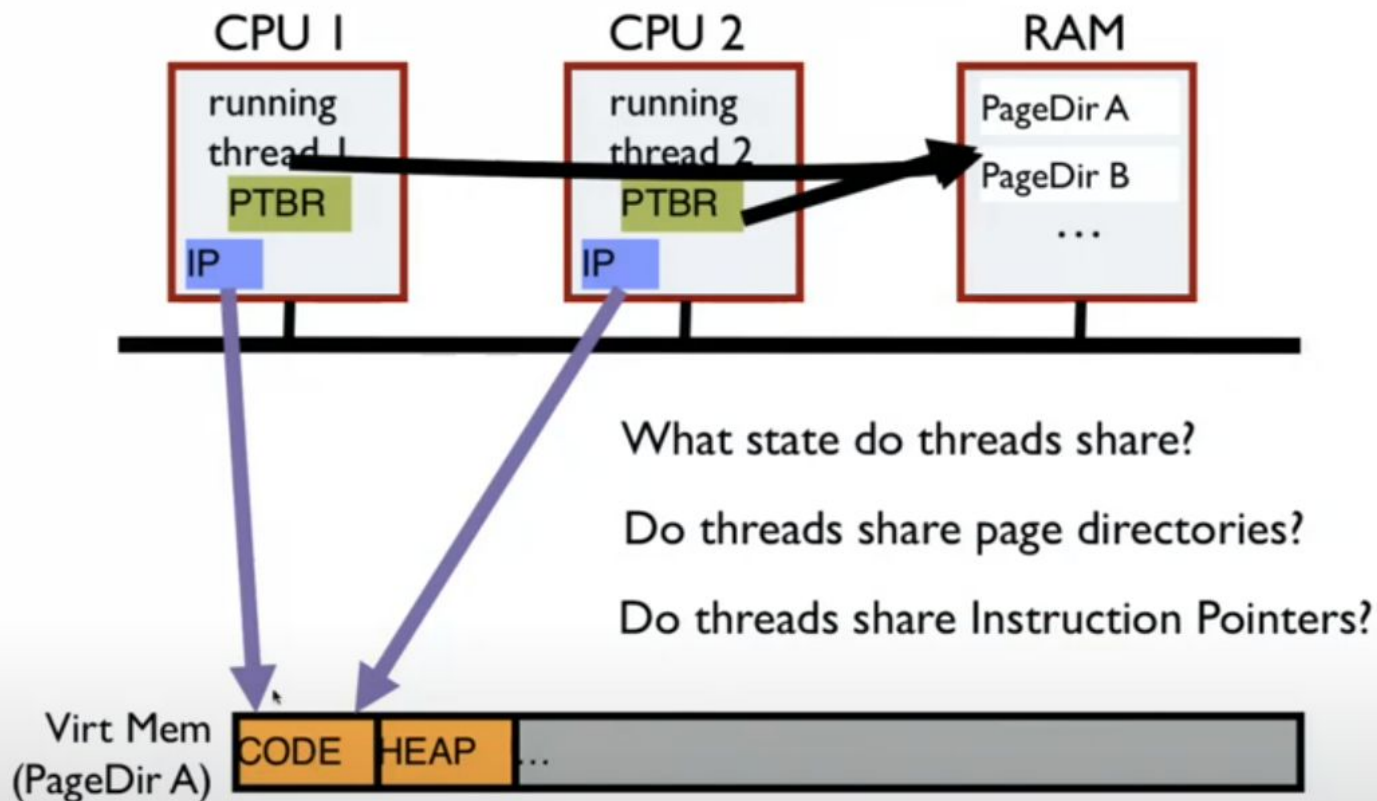
“own” -> but not really protected from other threads

Inter-thread communication is cheaper (than inter-process communication)

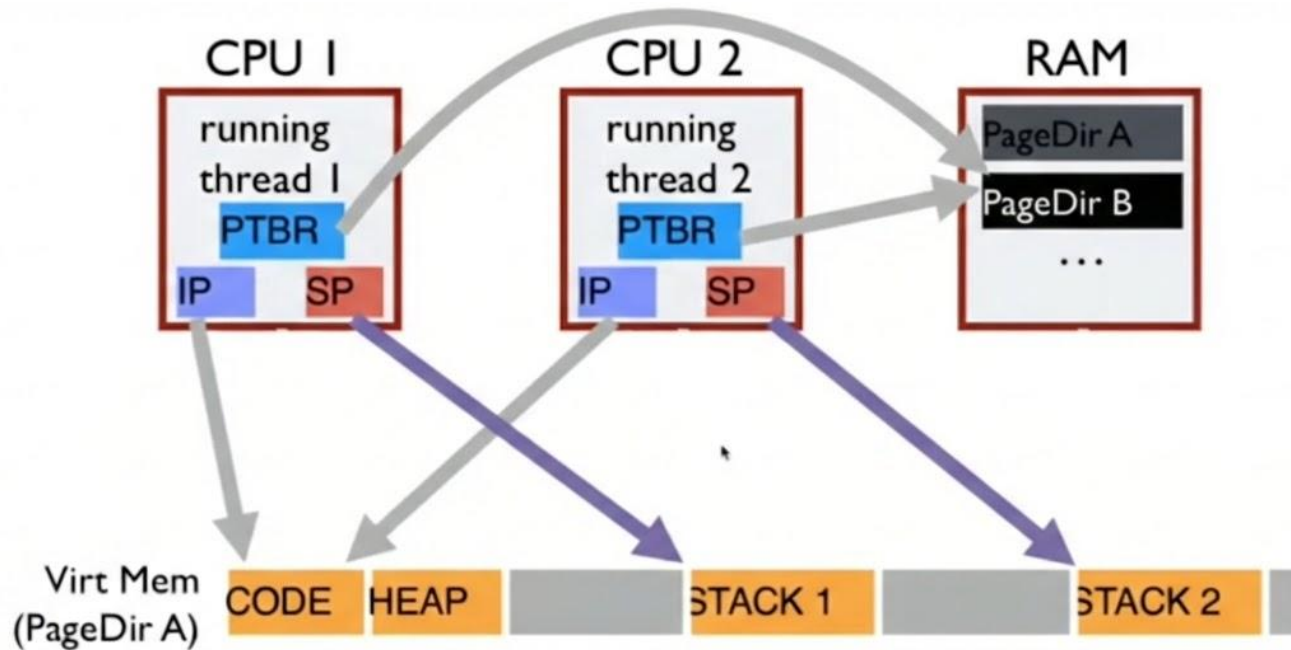
Don't need to send the actual data because the heap is shared

In this course: POSIX thread library, aka **pthread**s





Share code, but each thread may be executing different code at the same time
→ Different Instruction Pointers



Do threads share stack pointer?

Threads executing different functions need different stacks
(But, stacks are in same address space, so trusted to be cooperative)

THREADS WITHIN A PROCESS

All threads **share**:

- Process ID (PID)
- Page Table (PTBR), Address space: code (instructions), data (heap)
- Open file descriptors
- Current working directory
- User and group id

Each thread has its own **unique**:

- Thread ID (TID)
- Set of registers, including Program Counter and Stack Pointer
- Stack for local variables and return addresses (in same address space)

Threading: Approach 1

User-level threads: Many-to-one kernel thread mapping

- Implemented by user-level runtime libraries
- OS unaware of the existence of user-level threads
- OS believes each process contains only a single thread of control

Pros:

- No OS requirements; Completely portable
- Can tune policies to suit application
- Lower overhead switching; no system call to switch threads

Cons:

- Cannot leverage multiprocessors
- Entire process blocks when one thread blocks

Threading: Approach 2

Kernel-level threads: One-to-one thread mapping

- User-level runtime lib, but each user-level thread gets a kernel thread
- Each kernel thread schedulable independently
- OS performs thread operations (create, schedule, synchronize)

Pros:

- Multiple threads can run in parallel
- When a thread blocks, other threads are unaffected

Cons:

- Higher overhead for thread operations (compared to Approach 1)

THREAD SCHEDULE

```
volatile int balance = 0;
int loops = 1000000; // 1 million

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        balance++;
    }
    pthread_exit(NULL);
}
```

```
int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    printf("Initial value : %d\n", balance);
    pthread_create(&p1, NULL, worker, NULL);
    pthread_create(&p2, NULL, worker, NULL);
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf("Final value    : %d\n", balance);
    return 0;
}
```

EXPECTATION: Students will teach themselves pthreads

NEXT

More Concurrency!